

HDOC DAY-32

Associative Properties of Matrix Addition and Matrix Multiplication Using Variable Length Arrays

1. Algorithm

Main Algorithm

1. Start.
2. Declare `choice`.
3. Display the following menu:
 - **1. Verify associative property of matrix addition**
 - **2. Verify associative property of matrix multiplication**
 - **3. Exit**
4. Read the user's choice.
5. Use a `switch` statement:
 - If `choice = 1`, call the function for associative matrix addition.
 - If `choice = 2`, call the function for associative matrix multiplication.
 - If `choice = 3`, terminate the program.
 - Otherwise, display an invalid-choice message.
6. Repeat the menu until the user selects Exit.
7. Stop.

Algorithm for Associative Property of Matrix Addition

1. Read the number of rows m and columns n .
2. Declare three variable-length matrices $A[m][n]$, $B[m][n]$, and $C[m][n]$.
3. Read the elements of matrices A , B , and C .
4. Calculate $AB = A + B$.
5. Calculate $Left = AB + C$, representing $(A+B)+C$.
6. Calculate $BC = B + C$.
7. Calculate $Right = A + BC$, representing $A+(B+C)$.
8. Display $Left$ and $Right$.
9. Compare corresponding elements of both matrices.
10. If all elements are equal, display that the associative property is verified; otherwise, display that it is not verified.

Algorithm for Associative Property of Matrix Multiplication

1. Read m , n , p , and q .

2. Declare:
 - o $A[m][n]$
 - o $B[n][p]$
 - o $C[p][q]$
 3. Read matrices A, B, and C.
 4. Calculate $AB = A \times B$.
 5. Calculate $Left = AB \times C$, representing $(A \times B) \times C$.
 6. Calculate $BC = B \times C$.
 7. Calculate $Right = A \times BC$, representing $A \times (B \times C)$.
 8. Display both result matrices.
 9. Compare their corresponding elements.
 10. If all elements are equal, display that the associative property is verified; otherwise, display that it is not verified.
-

2. C Program

```
#include <stdio.h>

// Function to read a matrix
void readMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
}

// Function to display a matrix
void displayMatrix(int r, int c, int a[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            printf("%d\t", a[i][j]);
        }
        printf("\n");
    }
}

// Function to add two matrices
void addMatrix(int r, int c,
               int a[r][c],
```

```

        int b[r][c],
        int result[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            result[i][j] = a[i][j] + b[i][j];
        }
    }
}

// Function to compare two matrices
int compareMatrix(int r, int c,
                  int a[r][c],
                  int b[r][c])
{
    int i, j;

    for (i = 0; i < r; i++)
    {
        for (j = 0; j < c; j++)
        {
            if (a[i][j] != b[i][j])
            {
                return 0;
            }
        }
    }

    return 1;
}

// Function for associative property of matrix addition
void associativeAddition()
{
    int m, n;

    printf("Enter number of rows and columns: ");
    scanf("%d %d", &m, &n);

    int A[m][n], B[m][n], C[m][n];
    int AB[m][n], BC[m][n];
    int left[m][n], right[m][n];

    printf("Enter matrix A:\n");
    readMatrix(m, n, A);

    printf("Enter matrix B:\n");
    readMatrix(m, n, B);

    printf("Enter matrix C:\n");
    readMatrix(m, n, C);

    // (A + B) + C

```

```

addMatrix(m, n, A, B, AB);
addMatrix(m, n, AB, C, left);

// A + (B + C)
addMatrix(m, n, B, C, BC);
addMatrix(m, n, A, BC, right);

printf("\n(A + B) + C:\n");
displayMatrix(m, n, left);

printf("\nA + (B + C):\n");
displayMatrix(m, n, right);

if (compareMatrix(m, n, left, right))
{
    printf("\nAssociative property of matrix addition is verified.\n");
}
else
{
    printf("\nAssociative property of matrix addition is not
verified.\n");
}
}

// Function to multiply two matrices
void multiplyMatrix(int r1, int c1, int c2,
                    int a[r1][c1],
                    int b[c1][c2],
                    int result[r1][c2])
{
    int i, j, k;

    for (i = 0; i < r1; i++)
    {
        for (j = 0; j < c2; j++)
        {
            result[i][j] = 0;

            for (k = 0; k < c1; k++)
            {
                result[i][j] += a[i][k] * b[k][j];
            }
        }
    }
}

// Function for associative property of matrix multiplication
void associativeMultiplication()
{
    int m, n, p, q;

    printf("Enter m, n, p and q: ");
    scanf("%d %d %d %d", &m, &n, &p, &q);

    int A[m][n];
    int B[n][p];
    int C[p][q];

```

```

int AB[m][p];
int BC[n][q];

int left[m][q];
int right[m][q];

printf("Enter matrix A (%d x %d):\n", m, n);
readMatrix(m, n, A);

printf("Enter matrix B (%d x %d):\n", n, p);
readMatrix(n, p, B);

printf("Enter matrix C (%d x %d):\n", p, q);
readMatrix(p, q, C);

// (A x B) x C
multiplyMatrix(m, n, p, A, B, AB);
multiplyMatrix(m, p, q, AB, C, left);

// A x (B x C)
multiplyMatrix(n, p, q, B, C, BC);
multiplyMatrix(m, n, q, A, BC, right);

printf("\n(A x B) x C:\n");
displayMatrix(m, q, left);

printf("\nA x (B x C):\n");
displayMatrix(m, q, right);

if (compareMatrix(m, q, left, right))
{
    printf("\nAssociative property of matrix multiplication is
verified.\n");
}
else
{
    printf("\nAssociative property of matrix multiplication is not
verified.\n");
}
}

// Main function
int main()
{
    int choice;

    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Verify associative property of matrix addition\n");
        printf("2. Verify associative property of matrix multiplication\n");
        printf("3. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)

```

```

    {
        case 1:
            associativeAddition();
            break;

        case 2:
            associativeMultiplication();
            break;

        case 3:
            printf("Program terminated.\n");
            break;

        default:
            printf("Invalid choice! Please enter 1, 2 or 3.\n");
    }

    } while (choice != 3);

    return 0;
}

```

3. Test Case Table

Test Case	Choice	Input Condition	Expected Output
TC-1	1	Three 2×2 positive integer matrices	$(A+B)+C = A+(B+C)$, property verified
TC-2	1	Three 1×3 matrices	Both result matrices equal, property verified
TC-3	1	2×2 matrices containing negative numbers and zeros	Both result matrices equal, property verified
TC-4	2	$A(2 \times 2)$, $B(2 \times 2)$, $C(2 \times 2)$	$(A \times B) \times C = A \times (B \times C)$, property verified
TC-5	2	$A(2 \times 3)$, $B(3 \times 2)$, $C(2 \times 3)$	Both 2×3 result matrices equal, property verified
TC-6	2	2×2 matrices with B as zero matrix	Both sides produce zero matrix, property verified
TC-7	1	Three minimum-size 1×1 matrices	Both sides equal, property verified
TC-8	5	Invalid menu choice	Invalid-choice message displayed
TC-9	3	Exit	Program terminates successfully

4. Test Case Simulation and Evaluation

TC-1: Matrix Addition with 2×2 Matrices

```
(kali㉿kali)-[~/Desktop]
$ cc day32.c

(kali㉿kali)-[~/Desktop]
$ ./a.out

===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter number of rows and columns: 2 2
Enter matrix A:
1 2
3 4
Enter matrix B:
5 6
7 8
Enter matrix C:
9 0
1 2

(A + B) + C:
15      8
11      14

A + (B + C):
15      8
11      14

Associative property of matrix addition is verified.
```

Evaluation

Both sides produce the same matrix. Therefore, $(A+B)+C = A+(B+C)$ and the **associative property of matrix addition is verified**.

Result: PASS

TC-2: Matrix Addition with 1×3 Matrices

```

===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter number of rows and columns: 1 3
Enter matrix A:
1 2 3
Enter matrix B:
4 5 6
Enter matrix C:
7 8 9

(A + B) + C:
12 15 18

A + (B + C):
12 15 18

Associative property of matrix addition is verified.

```

Evaluation

Both sides produce $[12 \ 15 \ 18]$. Hence, the program correctly verifies the **associative property for a single-row matrix**.

Result: PASS

TC-3: Matrix Addition with Negative and Zero Elements

```

===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter number of rows and columns: 2 2
Enter matrix A:
0 -8
-31 0
Enter matrix B:
-5 0
0 -7
Enter matrix C:
0 0
0 0

(A + B) + C:
-5 -8
-31 -7

A + (B + C):
-5 -8
-31 -7

Associative property of matrix addition is verified.

```

Evaluation

Both result matrices are identical even with negative and zero elements. Hence, the **associative property of matrix addition is verified**.

Result: PASS

TC-4: Matrix Multiplication with 2×2 Matrices

```
===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m, n, p and q: 2 2 2 2
Enter matrix A (2 x 2):
1 2
3 4
Enter matrix B (2 x 2):
5 6
3 4
Enter matrix C (2 x 2):
8 7
9 0

(A x B) x C:
214 77
522 189

A x (B x C):
214 77
522 189

Associative property of matrix multiplication is verified.
```

Evaluation

Both sides produce the same 2×2 matrix. Hence, $(A \times B) \times C = A \times (B \times C)$ and the **associative property of matrix multiplication is verified**.

Result: PASS

TC-5: Matrix Multiplication with Different Dimensions

```
===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m, n, p and q: 2 3 2 3
Enter matrix A (2 x 3):
2 3 3
2 9 8
Enter matrix B (3 x 2):
3 4
36
4 9
0
Enter matrix C (2 x 3):
2 3 0
8 7 6
(A x B) x C:
442    563    120
1156   1514    264
A x (B x C):
442    563    120
1156   1514    264
Associative property of matrix multiplication is verified.
```

Evaluation

Both sides give exactly the same 2×3 matrix. Thus, the program verifies the **associative property of multiplication for matrices having different compatible dimensions**.

Result: PASS

TC-6: Matrix Multiplication with Zero Matrix

```

===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 2
Enter m, n, p and q: 2 2 2 2
Enter matrix A (2 x 2):
9 8
7 6
Enter matrix B (2 x 2):
0 0
0 0
Enter matrix C (2 x 2):
6 1
2 3

(A x B) x C:
0 0
0 0

A x (B x C):
0 0
0 0

Associative property of matrix multiplication is verified.

```

Evaluation

Both sides produce the same zero matrix. Hence, the **associative property of matrix multiplication is verified when one matrix is a zero matrix.**

Result: PASS

TC-7: Minimum Matrix Size 1×1

```

===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 1
Enter number of rows and columns: 1 1
Enter matrix A:
8
Enter matrix B:
31
Enter matrix C:
57

(A + B) + C:
96

A + (B + C):
96

Associative property of matrix addition is verified.

```

Evaluation

Both sides produce 9. Therefore, the program correctly handles the **minimum VLA matrix size of 1×1** and verifies the associative property.

Result: PASS

TC-8: Invalid Menu Choice

```
===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 5
Invalid choice! Please enter 1, 2 or 3.
```

Evaluation

The program successfully detects the invalid menu choice and displays an appropriate error message. The menu is displayed again for another choice.

Result: PASS

TC-9: Exit Option

```
===== MENU =====
1. Verify associative property of matrix addition
2. Verify associative property of matrix multiplication
3. Exit
Enter your choice: 3
Program terminated.
```

Evaluation

The program executes `case 3` and exits the `do-while` loop. Hence, the **Exit option works correctly and terminates the program successfully.**

Result: PASS

5. Final Test Case Evaluation

Test Case	Operation Tested	Actual Result	Status
TC-1	Addition – normal 2×2 matrices	Associative property verified	PASS
TC-2	Addition – 1×3 matrices	Associative property verified	PASS
TC-3	Addition – negative/zero elements	Associative property verified	PASS
TC-4	Multiplication – 2×2 matrices	Associative property verified	PASS
TC-5	Multiplication – different dimensions	Associative property verified	PASS
TC-6	Multiplication – zero matrix	Associative property verified	PASS
TC-7	Minimum 1×1 matrix	Associative property verified	PASS
TC-8	Invalid choice	Error handled correctly	PASS
TC-9	Exit	Program terminated correctly	PASS

6. Conclusion

The program was compiled, executed, and tested using the prepared test cases. All test cases produced the expected results.

Thus, the program successfully verifies the associative properties:

Matrix Addition:

$$(A + B) + C = A + (B + C)$$

Matrix Multiplication:

$$(A \times B) \times C = A \times (B \times C)$$

The program also correctly handles **variable-length arrays**, **different compatible matrix dimensions**, **zero/negative values**, **invalid menu choices**, and **program termination**.